Student Records Portal (ICS570 Lab 2)

Passkey (WebAuthn/FIDO2) sign-in, server-side role and ownership checks, and security tests T1–T8. Fictional data only.

Stack: Node.js 22, Express 5, express-session (server-side sessions), `@simplewebauthn/server` and `/browser` 14, SQLite via Node's built-in `node:sqlite`, `playwright-core` for the test runner. RP ID `localhost`, origin `http://localhost:3000`.

Prerequisites

- Node.js 22.13 or newer (`node -v`; with nvm, `nvm use` reads `.nvmrc`). Nothing to compile.
- Google Chrome on macOS with Touch ID or a device password (Safari also works for the manual tests).

Install and run

```
npm ci                    # exact versions from package-lock.json
cp .env.example .env      # set SESSION_SECRET to a long random string
npm run seed              # builds data/portal.db, prints one-time codes
npm start                 # http://localhost:3000 (not 127.0.0.1)
```

Accounts: **S001**, **S002** (students in C101; S002 also takes C102), **L001** (lecturer, C101), **L002** (lecturer, C102). Codes are single use and expire after 24 hours. To start again: stop the server, `npm run seed`, `npm start`.

Repeat T1–T8 by hand (Chrome; DevTools open, Network tab with **Preserve log** ticked)

Test	Steps	Expected
T1	**First time here? Set up a passkey**: enter `S001` and its code, **Check code**, **Create passkey**, confirm with Touch ID	"Passkey saved for S001"
T2	Enter `S001`, **Sign in with passkey**, confirm	S001's record opens
T3	In Network, right-click the T2 `login/verify` request, **Copy as fetch**. **Sign out**, click **Sign in with passkey** and cancel the prompt. In Console (type `allow pasting` if asked), paste and append `.then(r => r.json()).then(console.log)`	400, challenge mismatch; `/me` is 401
T4	Private window: `http://localhost:3000/students/S001`	401
T5	Signed in as S001: the record page, or `/students/S001`	200
T6	As S001, open `/students/S002`; in Console run `await fetch('/students/S002').then(r => r.status)`	403
T7	As S001, in Console: `await fetch('/courses/C101/grades/S002', {method:'POST', headers:{'Content-Type':'application/json'}, body:'{"grade":"E"}'}).then(r => r.status)`	403; signed in as L001 later, S002 still shows A
T8	Sign out, set up and sign in as `L001`, change S001's C101 grade, **Save grade**	"Saved: B+ to A"

Automated evidence run

Stop `npm start` (both use port 3000), then `npm run evidence`. It runs T1–T8 and seven extra checks (X1–X7) in Chrome against a separate database (`data/evidence.db`). Chrome's virtual authenticator stands in for Touch ID. Results go to `evidence/`: `results.md`, labelled screenshots, `requests.log`, `server.log`, `audit-log.txt`. Add `HEADED=1` to watch.

Other commands

`npm run admin -- unassign L001 C101` ends a lecturer's course access; `issue-code S001` issues a recovery code; `revoke-passkeys S001` revokes passkeys and ends sessions. `npm run partb` runs the AES-GCM follow-on.

Where the security decisions live

`server.js`: `requireAuth` (authentication gate), enrollment, WebAuthn routes, sessions, logging. `policy.js`: every authorization rule, deny by default. `db.js`: schema.